Centroid Dekompozíció és Heavy-Light Dekompozíció

Szerző: Nikházy László
Előadó: Gyimesi Péter

2026. március 26.

Források:
CP-Algorithms
USACO Guide
Medium

Xenia and Tree (Codeforces 342E) – Feladat

Adott egy fa n csúccsal. Kezdetben az 1-es csúcs piros, a többi kék.

Feladat: q darab lekérdezést kell feldolgoznunk, amelyek kétfélék lehetnek:

1. Egy megadott kék csúcs pirosra színezése.
2. A legközelebbi piros csúcs távolságának meghatározása egy adott csúcstól.

Korlátok:

- $2 \leq n \leq 10^5$, $1 \leq q \leq 10^5$

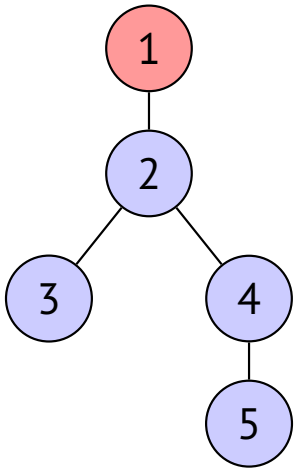
Bemenet: Az első sor tartalmazza n és q értékét, a következő $n-1$ sor a fa éleit. Ezt követi q lekérdezés: 1 v (színezés) vagy 2 v (távolság).

Példa bemenet:

```
5 4
1 2
2 3
2 4
4 5
2 1
2 5
1 2
2 5
```

Kimenet:

```
0
3
2
```



Lekérdezések:

2 1 → 0
2 5 → 3
1 2 → (2-es piros)
2 5 → 2

Xenia and Tree – Naiv megoldások

1. **Keresés a lekérdezésnél (BFS)**
   - **Frissítés (színezés):** Csak átállítunk egy `is_red[v] = true` jelzőt. Lépésszám: $O(1)$.
   - **Lekérdezés:** Indítunk egy szélességi bejárást (BFS) az adott u csúcsból az első piros csúcsig. Lépésszám: $O(n)$.
   - **Probléma:** Ha majdnem minden lekérdezésünk távolságkeresés, a futásidő $O(n \cdot q)$ lesz.
2. **Távolságok folyamatos karbantartása**
   - Minden csúcsra eltároljuk a legközelebbi piros csúcstól (kezdetben az 1-estől) való távolságot egy `dist` tömbben.
   - **Lekérdezés:** Csak kiolvassuk és kiírjuk a `dist[u]` értékét. Lépésszám: $O(1)$.
   - **Frissítés (színezés):** Ha egy v csúcs piros lesz, indítunk belőle egy BFS-t, és minden u csúcsra frissítjük az értéket: `dist[u]` = `min(dist[u],` távolság`(v, u))`. Lépésszám: $O(n)$.
   - **Probléma:** Ha sok a színezés, a futásidő szintén $O(n \cdot q)$ lesz.

Konklúzió: Mivel $n, q \leq 10^5$, az $O(n \cdot q)$ lépésszám (akár 10^{10} művelet) túl lassú. Valami okosabbra van szükségünk!

Ciel the Commander (Codeforces 321C) – Feladat

Adott egy fa $\mathtt{N}$ csúccsal. A feladatunk az, hogy a fa minden csúcsához rendeljünk egy betűt az angol ábécéből ('A'-tól 'Z'-ig, ahol az 'A' a legmagasabb szintű betű).

Feltétel: Ha a fában két különböző csúcs **ugyanazt a betűt** kapja, akkor a köztük lévő egyszerű úton lennie kell legalább egy **szigorúan magasabb** (az ábécében előrébb lévő) betűvel rendelkező csúcsnak.

Korlátok:

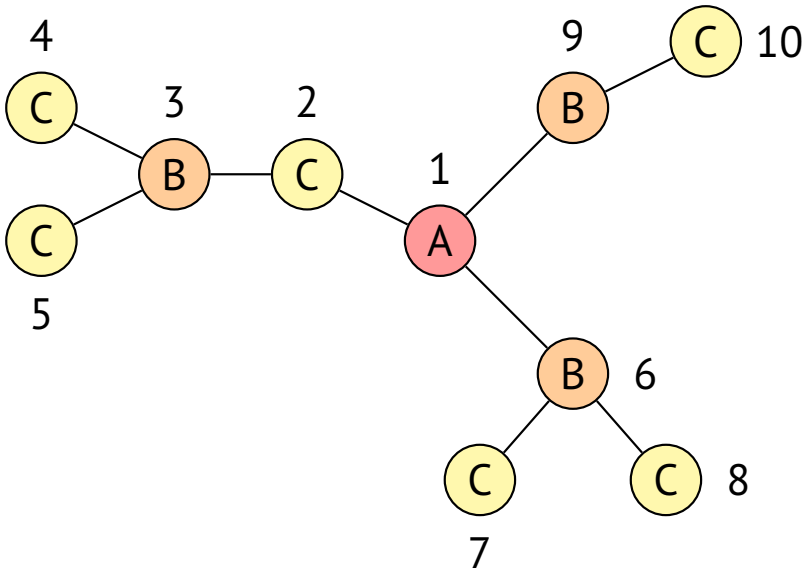
- $2 \leq \mathtt{N} \leq 10^5$

Példa bemenet:

```
10
1 2
2 3
3 4
3 5
1 6
6 7
6 8
1 9
9 10
```

Kimenet:

A C B C C B C C B C



Előző feladat a betűzés segítségével

Tegyük fel, hogy sikeresen megbetűzhetünk egy tetszőleges fát (A-Z). Ekkor érvényes egy remek tulajdonság:

Bármely két csúcs (`u` és `v`) közötti úton van egy egyértelmű "legmagasabb rangú" csúcs (legyen ez `X`).

Emiatt az $u \to v$ út mindig felbontható két részre: $u \to X$ és $X \to v$.
Tehát a távolságuk: `dist(u, v) = dist(u, X) + dist(X, v)`.

Hogyan oldja ez meg a Xenia and Tree feladatot?

Előző feladat a betűzés segítségével

Tegyük fel, hogy sikeresen megbetűzhetünk egy tetszőleges fát (A-Z). Ekkor érvényes egy remek tulajdonság:

Bármely két csúcs (`u` és `v`) közötti úton van egy egyértelmű "legmagasabb rangú" csúcs (legyen ez `X`).

Emiatt az `u` $\rightarrow$ `v` út mindig felbontható két részre: `u` $\rightarrow$ `X` és `X` $\rightarrow$ `v`.
Tehát a távolságuk: `dist(u, v) = dist(u, X) + dist(X, v)`.

Hogyan oldja ez meg a Xenia and Tree feladatot?

- Minden csúcsnak van legfeljebb 26 "főnöke" (nála magasabb rangú csúcs, amit elér anélkül, hogy még magasabba ütközne, önmagát is beleértve).
- Minden `X` csúcsnál tartunk egy `best[X]` értéket: *milyen messze van a legközelebbi piros csúcs, amihez vezető úton csak* `X`*-nél alacsonyabb rangú csúcsok vannak?* (Ha nincs ilyen, akkor ∞).

Előző feladat a betűzés segítségével

Tegyük fel, hogy sikeresen megbetűzhetünk egy tetszőleges fát (A-Z). Ekkor érvényes egy remek tulajdonság:

Bármely két csúcs (`u` és `v`) közötti úton van egy egyértelmű "legmagasabb rangú" csúcs (legyen ez `X`).

Emiatt az $u \to v$ út mindig felbontható két részre: $u \to X$ és $X \to v$.
Tehát a távolságuk: `dist(u, v) = dist(u, X) + dist(X, v)`.

Hogyan oldja ez meg a Xenia and Tree feladatot?

- Minden csúcsnak van legfeljebb 26 "főnöke" (nála magasabb rangú csúcs, amit elér anélkül, hogy még magasabba ütközne, önmagát is beleértve).
- Minden `X` csúcsnál tartunk egy `best[X]` értéket: *milyen messze van a legközelebbi piros csúcs, amihez vezető úton csak `X`-nél alacsonyabb rangú csúcsok vannak?* (Ha nincs ilyen, akkor ∞).

A műveletek logikája (legfeljebb 26 lépés / lekérdezés):

1. **Színezés (`u` piros lesz):** Végigmegyünk `u` összes `X` "főnökén" (beleértve önmagát is). Frissítjük `X` rekordját:
   `best[X] = min(best[X], dist(u, X))`
2. **Lekérdezés (`u`-ból):** A legközelebbi piros csúcshoz vezető út biztosan érinti `u` valamelyik `X` főnökét. Végigmegyünk rajtuk, és keressük a minimumot:
   $\texttt{valasz} = \min_X(\texttt{dist(u, X)} + \texttt{best[X]})$

Fixed-Length Paths I (CSES 2080) – Feladat

Adott egy fa n csúccsal és egy k egész szám.

Feladat: Számoljuk meg, hány olyan egyszerű út van a fában, amely pontosan k élet tartalmaz!

Korlátok:

- $1 \leq n, k \leq 2 \cdot 10^5$

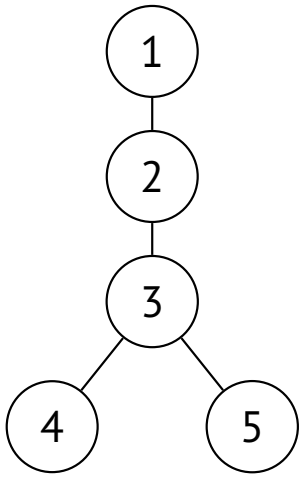
Bemenet: Az első sor tartalmazza n és k értékét. A következő $n - 1$ sor mindegyike egy-egy élet ír le (u, v).

Példa bemenet:

```
5 2
1 2
2 3
3 4
3 5
```

Kimenet:

4



Utak ($k = 2$):
1-2-3
2-3-4
2-3-5
4-3-5

Megoldás a betűzés segítségével

Itt is a csodafegyvert használjuk: **minden útnak van egy egyértelmű "legmagasabb rangú" csúcsa (legyen ez X).**

Egy k hosszú út, amelynek X a legmagasabb rangú csúcsa két részre bomlik X-en keresztül: egy d_1 és egy d_2 hosszú szakaszra, ahol $d_1 + d_2 = k$.

A számlálás menete minden X csúcsra:

- X-ből kiindulva megvizsgáljuk azokat az ágakat (részfákat), amelyek csak X-nél alacsonyabb rangú csúcsokból állnak.
- Létrehozunk egy `cnt` tömböt, ahol `cnt[d]` azt tárolja, hány darab `d` távolságra lévő csúcsot láttunk **eddig**. Kezdetben `cnt[0]` = 1, a többi 0.
- **Áganként haladva:**
  1. Megkeressük az ágban lévő összes csúcs távolságát X-től. Ha egy csúcs távolsága `d`, akkor az eddigi ágakból `cnt[k - d]` darab megfelelő párt tudunk alkotni vele. Ezeket hozzáadjuk a végeredményhez.
  2. Miután az egész ágat lekértük, frissítjük a `cnt` tömböt az ágban talált távolságokkal (így nem számolunk egy ágon belüli utakat).

Miért gyors ez?
Mivel minden csúcs legfeljebb 26 különböző X főnökhöz tartozik beosztottként, egy csúcsot az egész algoritmus során legfeljebb 26-szor járunk be.

A betűzés felépítése: Az alapötlet

Láttuk, milyen hasznos ez a betűzés. De hogyan tudjuk előállítani magát a hierarchiát egy tetszőleges fán?

Az "Oszd meg és uralkodj" stratégia:

- **1. lépés:** Válasszuk ki a fa "nagyfőnökét", és adjuk neki az **'A'** betűt.
- **2. lépés:** Képzeletben távolítsuk el ezt a csúcsot! Ekkor a fánk szétesik több kisebb, egymástól független részfára (komponensre).
- **3. lépés:** Ezekben a kisebb fákban ismételjük meg az eljárást **rekurzívan**: válasszuk ki az ő "főnöküket" (ők kapják a **'B'**-t), vegyük ki őket, és így tovább az ábécében.

A fő probléma: Az angol ábécé mindössze 26 betűből áll. Ez azt jelenti, hogy a rekurziónk **mélysége legfeljebb 26** lehet, mielőtt elfogynak a betűk! (És pont ettől lesznek a feladatoknál is olyan gyorsak a lekérdezéseink).

Kérdés:

Hogyan (milyen stratégia alapján) válasszuk ki minden lépésben a főnököt, hogy garantáljuk, a rekurzió sosem lesz túl mély?

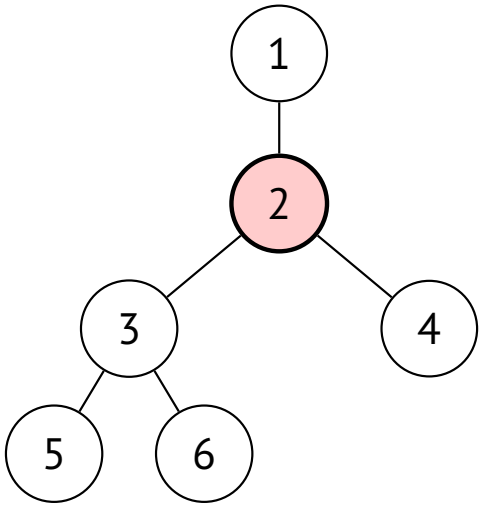
A centroid

Definíció: Egy $\mathtt{N}$ csúcsú fa **centroidja** (súlypontja) egy olyan csúcs, amelynek eltávolítása után a keletkező összefüggő komponensek (részfák) mindegyikének mérete legfeljebb $\mathtt{N}/2$.

Tulajdonságok:

- Minden fának legalább egy, de legfeljebb két centroidja van.
- Ha két centroid van, azok szomszédosak.
- A centroid a fa "közepe" a méret szempontjából, nem a távolságokéból (nem azonos az átmérő felezőpontjával).

Miért hasznos? Ha a centroidnál vágjuk szét a fát, garantáltan "kiegyensúlyozott" darabokat kapunk, ami lehetővé teszi az *oszd meg és uralkodj* stratégia alkalmazását.



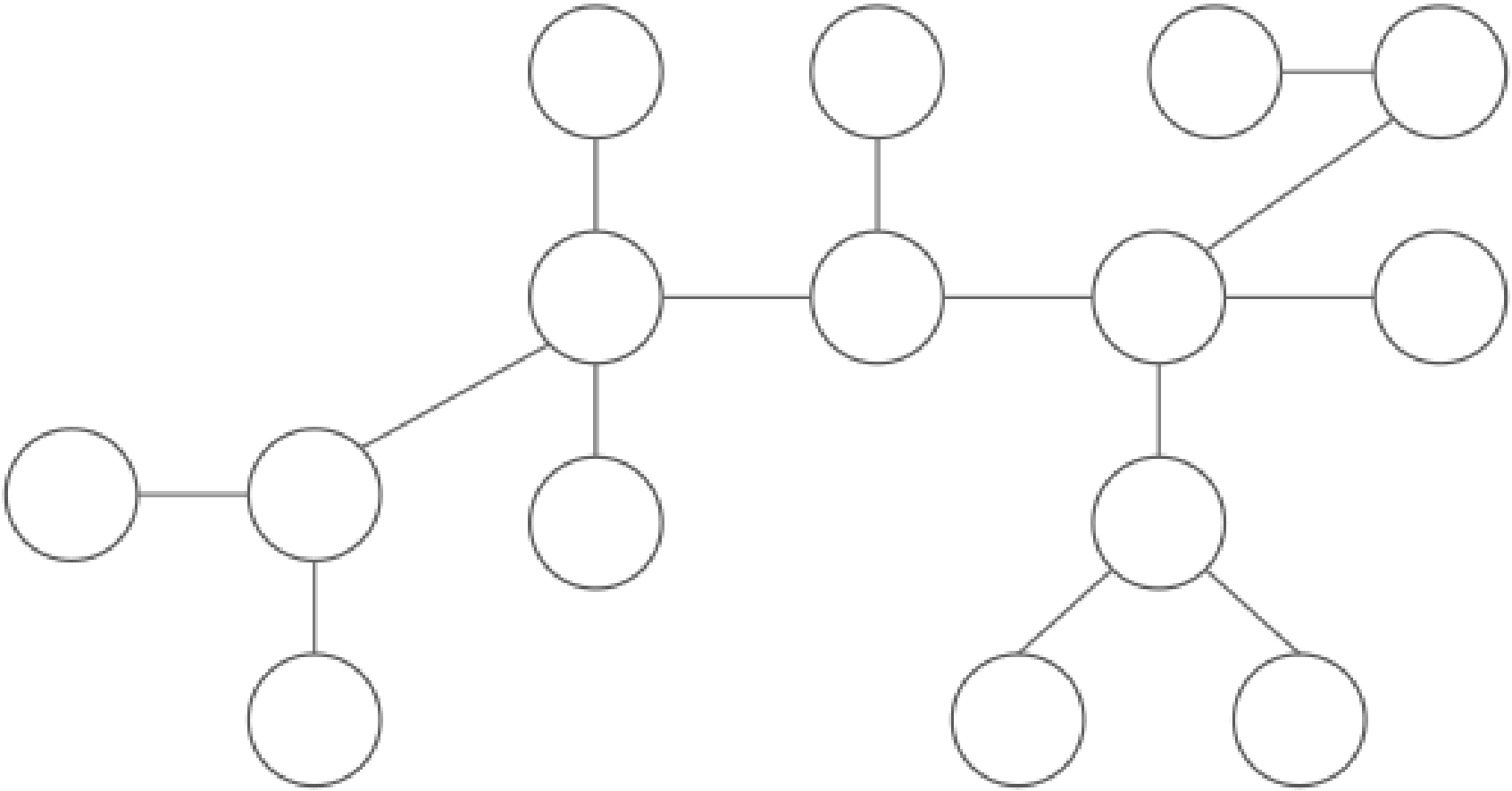
A centroid megkeresése

Algoritmus:

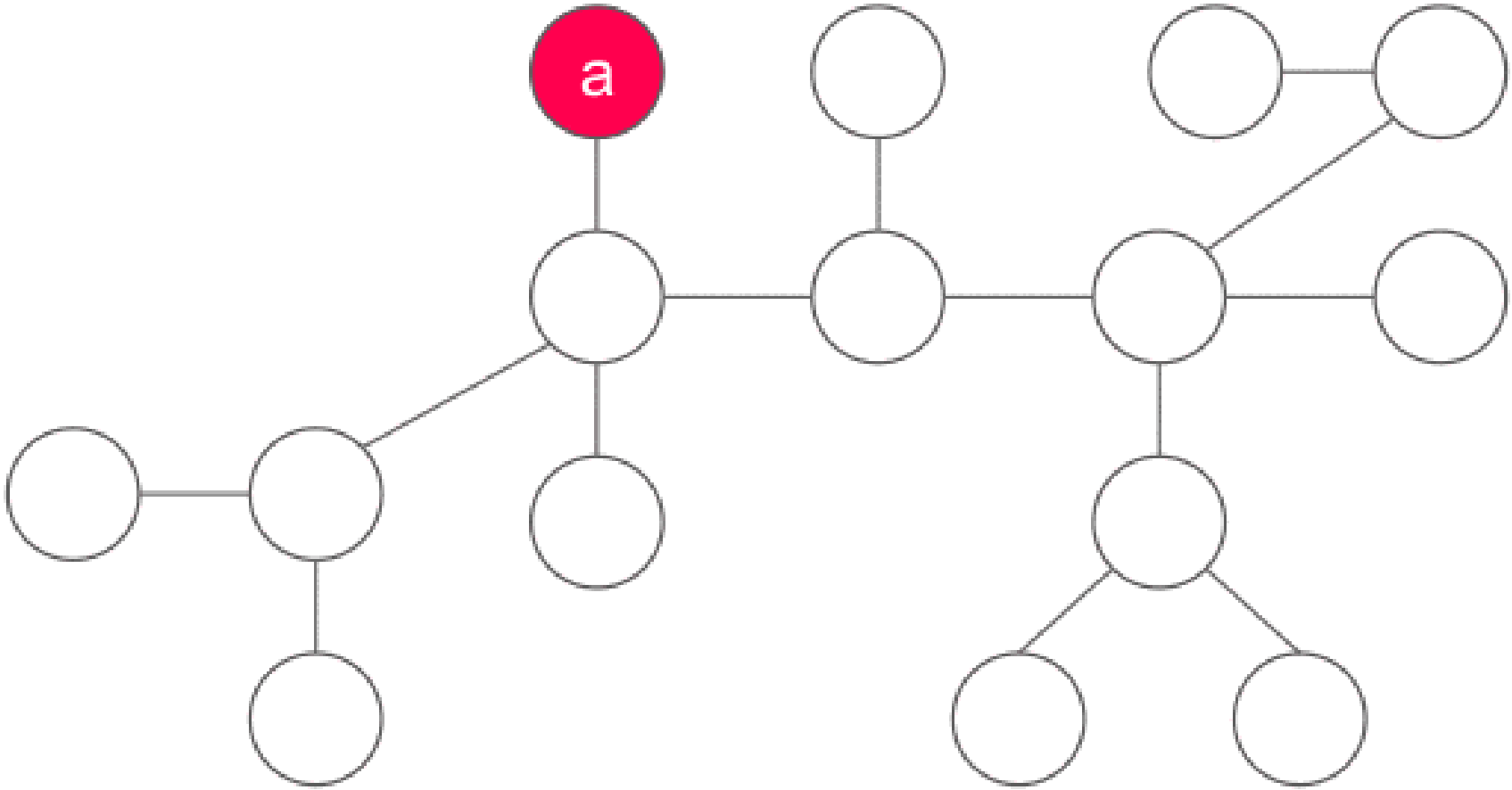
1. Válasszunk egy tetszőleges gyökeret.
2. Számítsuk ki minden csúcsra a hozzá tartozó részfa méretét (`subtree_size[u]`) egy DFS segítségével.
3. Induljunk el a gyökérből:
   - Ha van olyan `v` szomszéd, amelyre `subtree_size[v]` > N/2, lépjünk át `v`-be.
   - Ha nincs ilyen szomszéd, az aktuális csúcs a centroid.

Komplexitás: `O(N)`, mivel két DFS bejárás elegendő a méretek kiszámításához és a kereséshez.

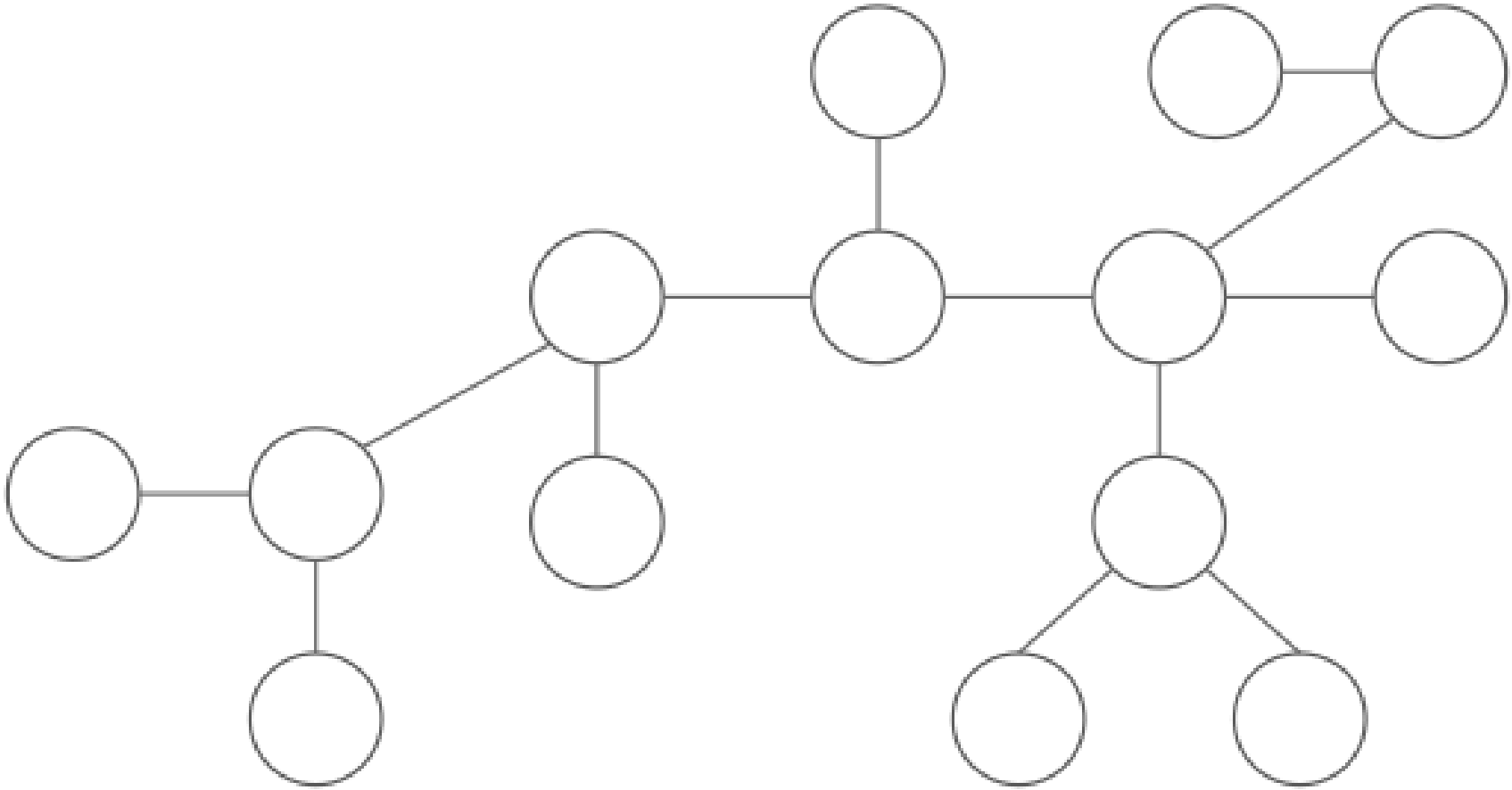
A centroid megkeresése - szemléltetés



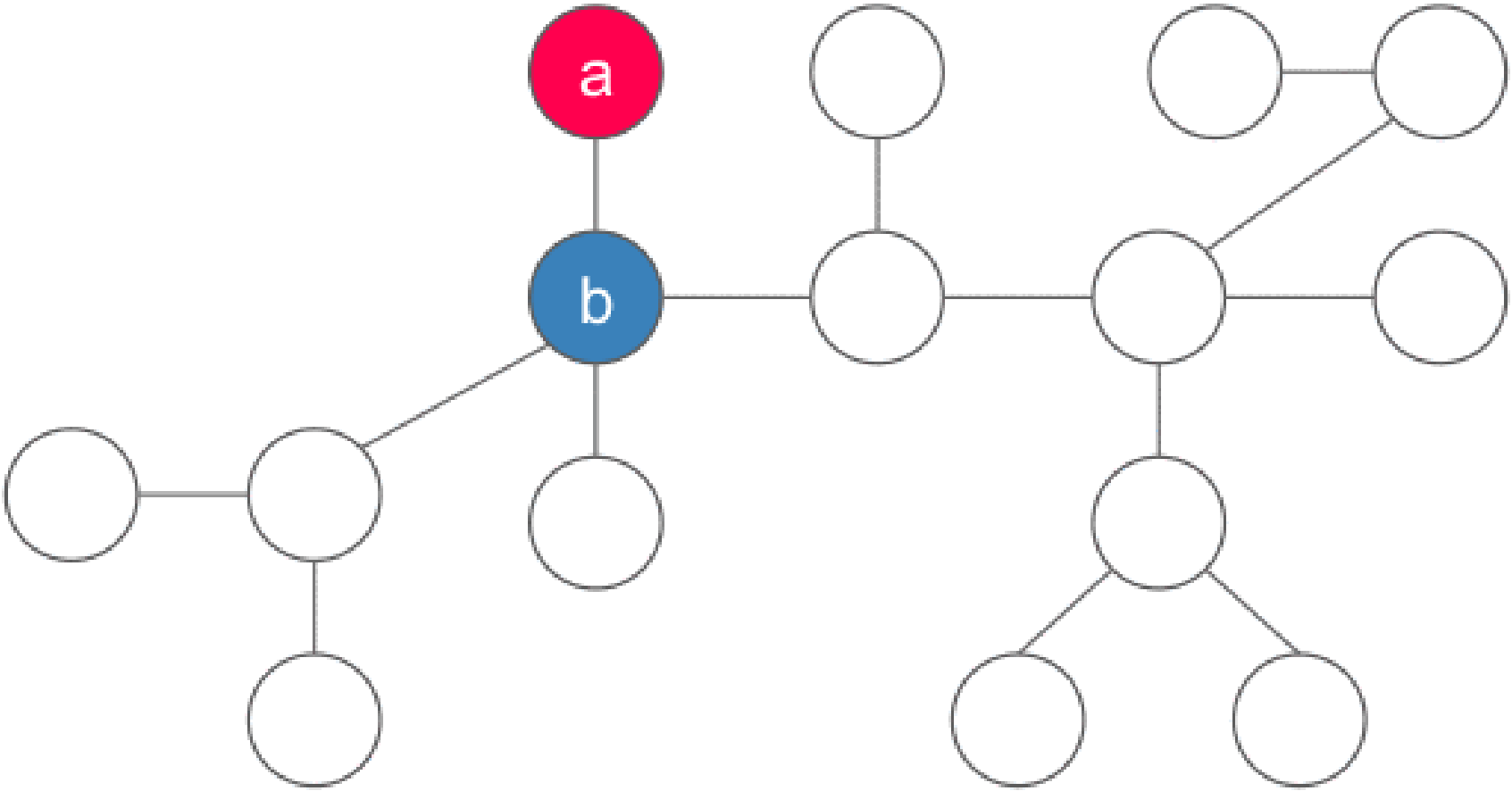
A centroid megkeresése - szemléltetés



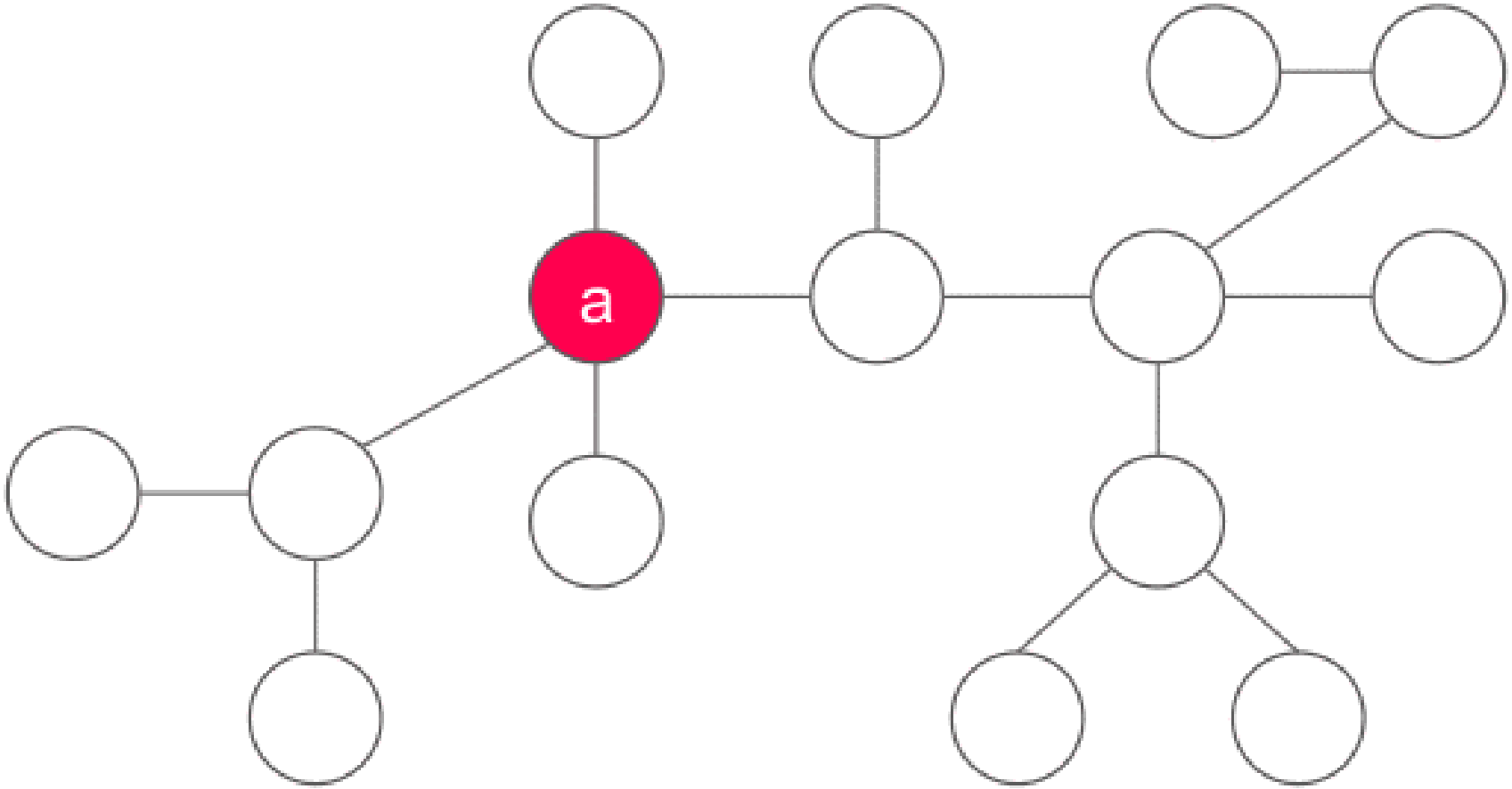
A centroid megkeresése - szemléltetés



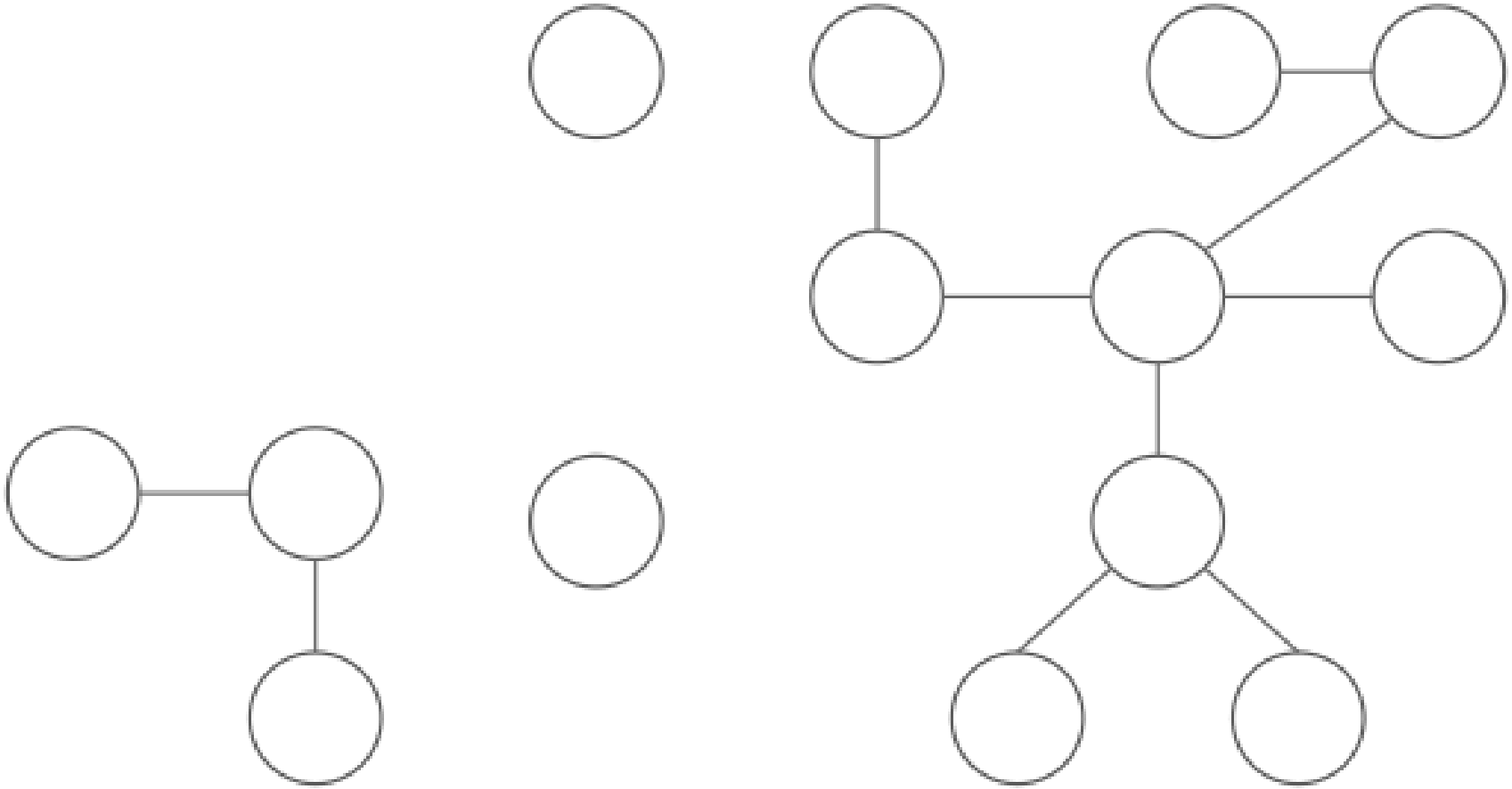
A centroid megkeresése - szemléltetés



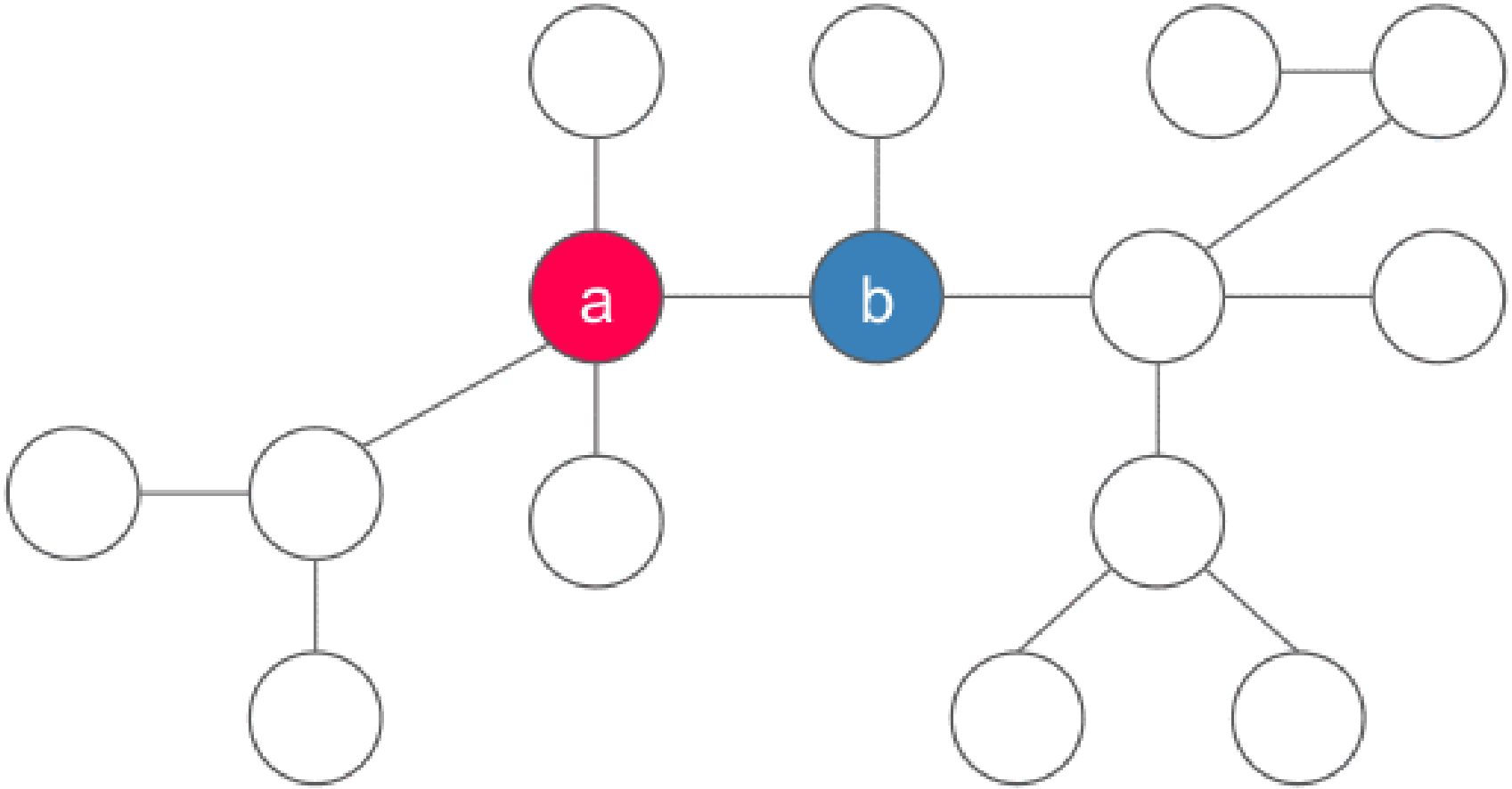
A centroid megkeresése - szemléltetés



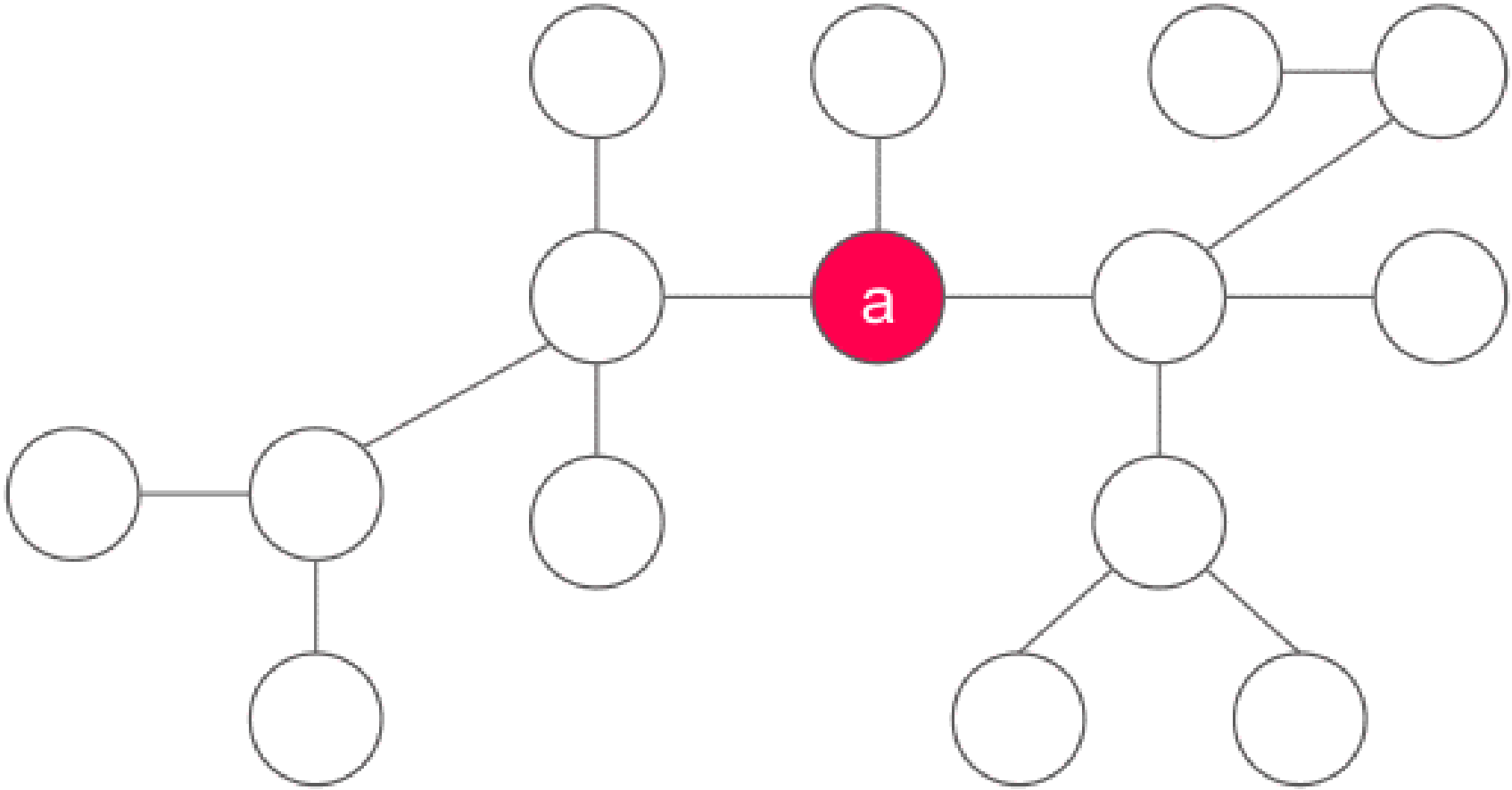
A centroid megkeresése - szemléltetés



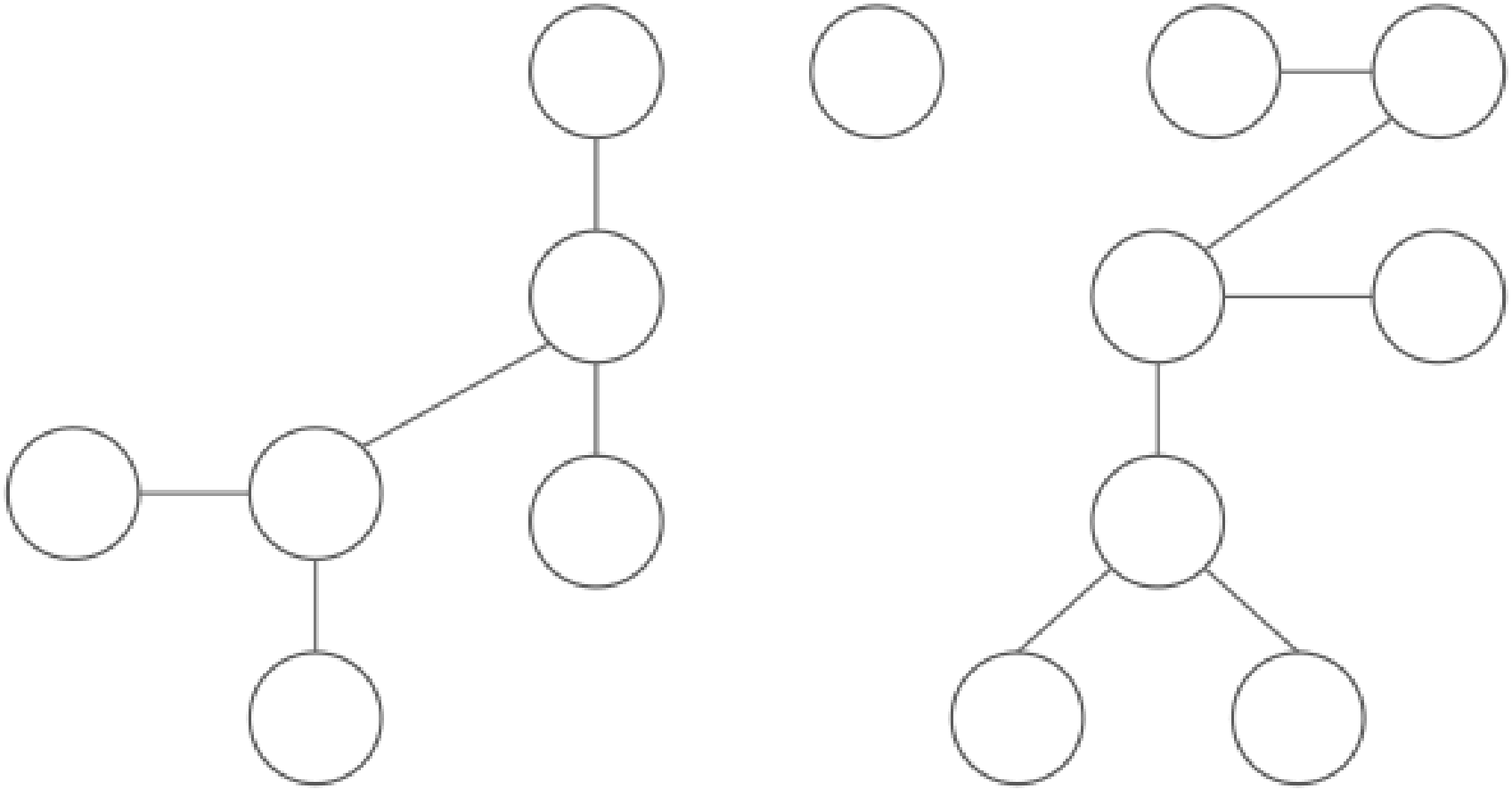
A centroid megkeresése - szemléltetés



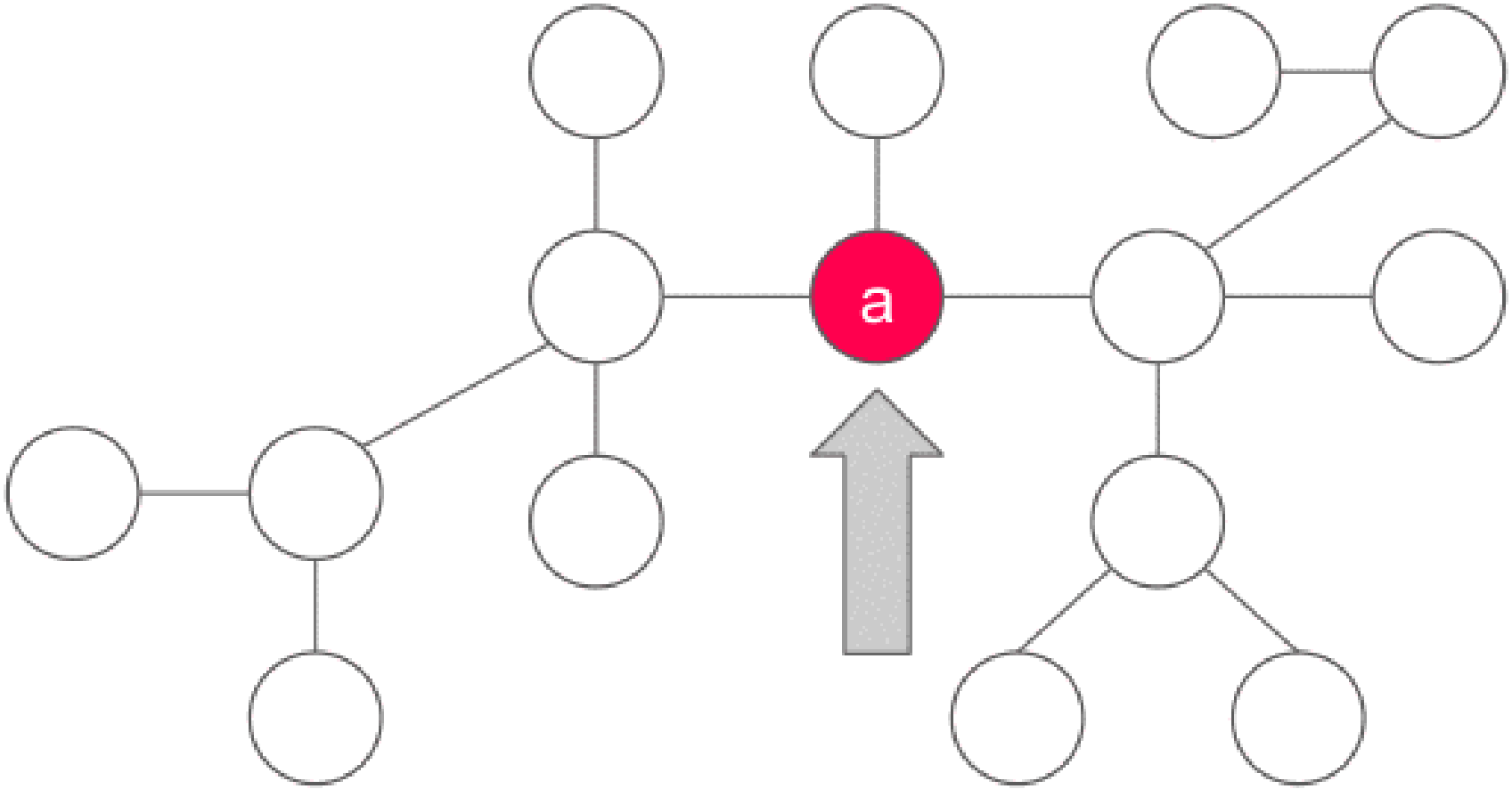
A centroid megkeresése - szemléltetés



A centroid megkeresése - szemléltetés



A centroid megkeresése - szemléltetés



Centroid keresése — Implementáció

```
int n;
vector<int> adj[maxn];
int subtree_size[maxn];

// Reszfa meretek meghatarozasa (DFS)
int get_subtree_size(int u, int p = -1) {
    subtree_size[u] = 1;
    for (int v : adj[u])
        if (v != p)
            subtree_size[u] += get_subtree_size(v, u);
    return subtree_size[u];
}

// A centroid megkeresese
int get_centroid(int u, int p = -1) {
    for (int v : adj[u])
        if (v != p && subtree_size[v] * 2 > n)
            // Ha egy szomszedos reszfa merete > n/2, abba az iranyba megyunk tovabb
            return get_centroid(v, u);
    return u; // Megtalaltuk a centroidot
}
```

Centroid dekompozíció

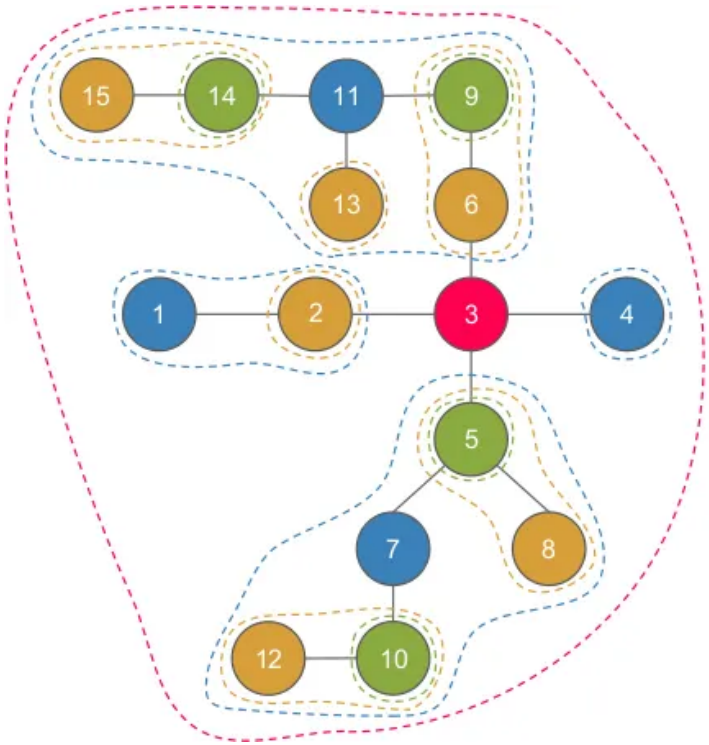
A fa centroid dekompozíciója egy másik fa (az ún. **centroid fa**), amelyet rekurzívan definiálunk:

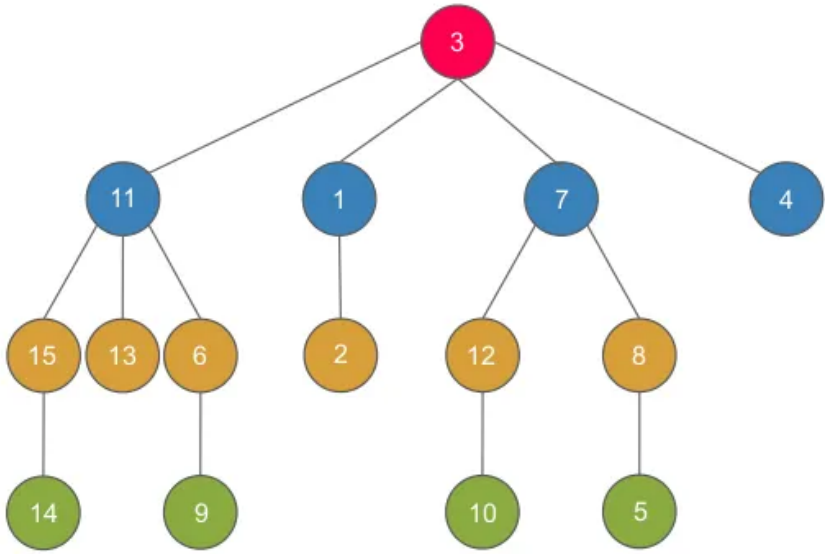
Definíció:

- A centroid fa **gyökere** az eredeti fa centroidja.
- A gyökér **gyermekei** azok a centroidok, amelyeket az eredeti centroid eltávolítása után kapott részfákból (komponensekből) kapunk rekurzív módon.

Vizuális különbség: Míg az eredeti fa tetszőlegesen "megnyúlt" lehet, a centroid dekompozícióval kapott fa magassága garantáltan **logaritmikus** (`O(log N)`).

Centroid dekompozíció — Szemléltetés





Egy eredeti fa (balra) és a belőle felépített centroid fa (jobbra). Megfigyelhető, hogy a centroid fa "laposabb", ami az algoritmus hatékonyságának kulcsa.

Időkomplexitás elemzése

Kérdés: Mennyi időbe telik felépíteni a struktúrát?

Elemzés:

- Első ránézésre $O(N^2)$-nek tűnhet, hiszen minden csúcs egyszer centroid lesz, és minden centroidnál lefuttatunk egy DFS-t.
- Valójában a komplexitás $O(N \log N)$.

Intuíció (Merge Sort analógia):

- Az első szinten $O(N)$ munkával megtaláljuk a fő centroidot.
- A második szinten több kisebb fában keresünk centroidot, de ezek méretének összege összesen $N - 1$. Ez is $O(N)$ munka.
- Mivel minden szinten a fa mérete legalább feleződik, összesen $\log N$ szintünk lesz.
- Szintenként $O(N)$ munka $\times$ $\log N$ szint = $O(N \log N)$.

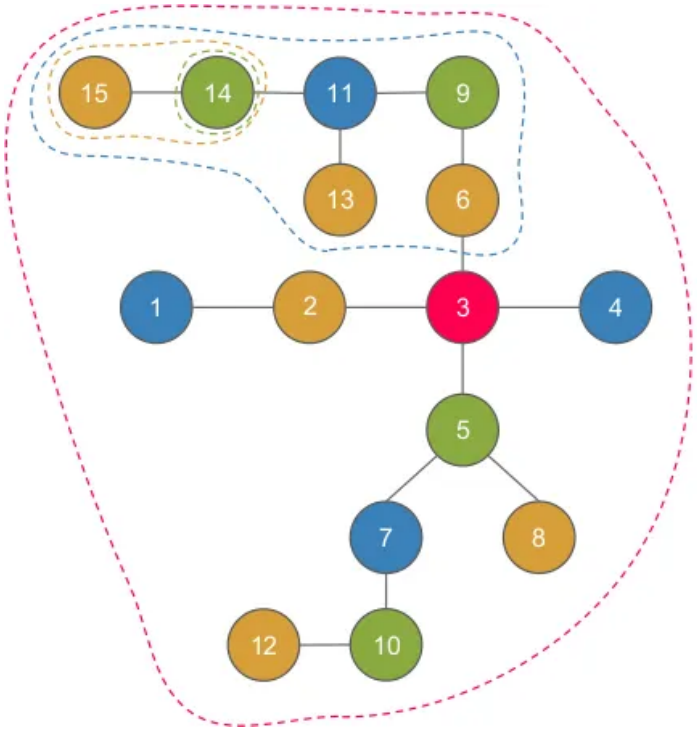
Implementáció

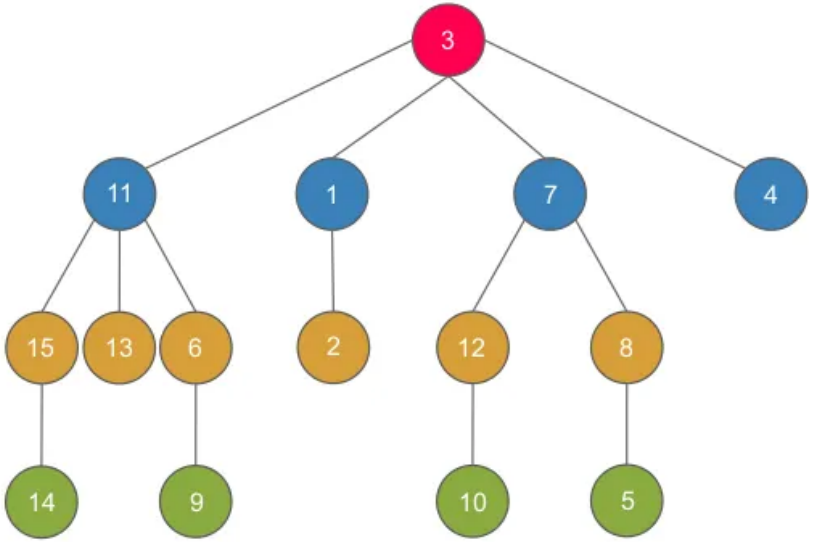
```
1   vector<vector<int>> adj;
2   vector<bool> is_removed;
3   vector<int> subtree_size;
4
5   int get_subtree_size(int node, int parent = -1) {
6     subtree_size[node] = 1;
7     for (int child : adj[node]) {
8       if (child == parent || is_removed[child]) { continue; }
9       subtree_size[node] += get_subtree_size(child, node);
10    }
11    return subtree_size[node];
12  }
13
14  int get_centroid(int node, int tree_size, int parent = -1) {
15    for (int child : adj[node]) {
16      if (child == parent || is_removed[child]) { continue; }
17      if (subtree_size[child] * 2 > tree_size) {
18        return get_centroid(child, tree_size, node);
19      }
20    }
21    return node;
22  }
23
24  void build_centroid_decomp(int node = 0) {
25    int centroid = get_centroid(node, get_subtree_size(node));
26    is_removed[centroid] = true;
27    for (int child : adj[centroid]) {
28      if (is_removed[child]) { continue; }
29      build_centroid_decomp(child);
30    }
31  }
```

Tulajdonság 1: Ősök és komponensek

Állítás: Egy csúcs minden centroid-fabeli ősének komponenséhez tartozik.

- Ha a centroid fában `b` az `a` őse, az azt jelenti, hogy `a` benne volt abban a részfában, aminek `b` volt a centroidja a dekompozíció során.
- Ezért `a` "elérhető" `b`-ből a dekompozíció adott fázisában.



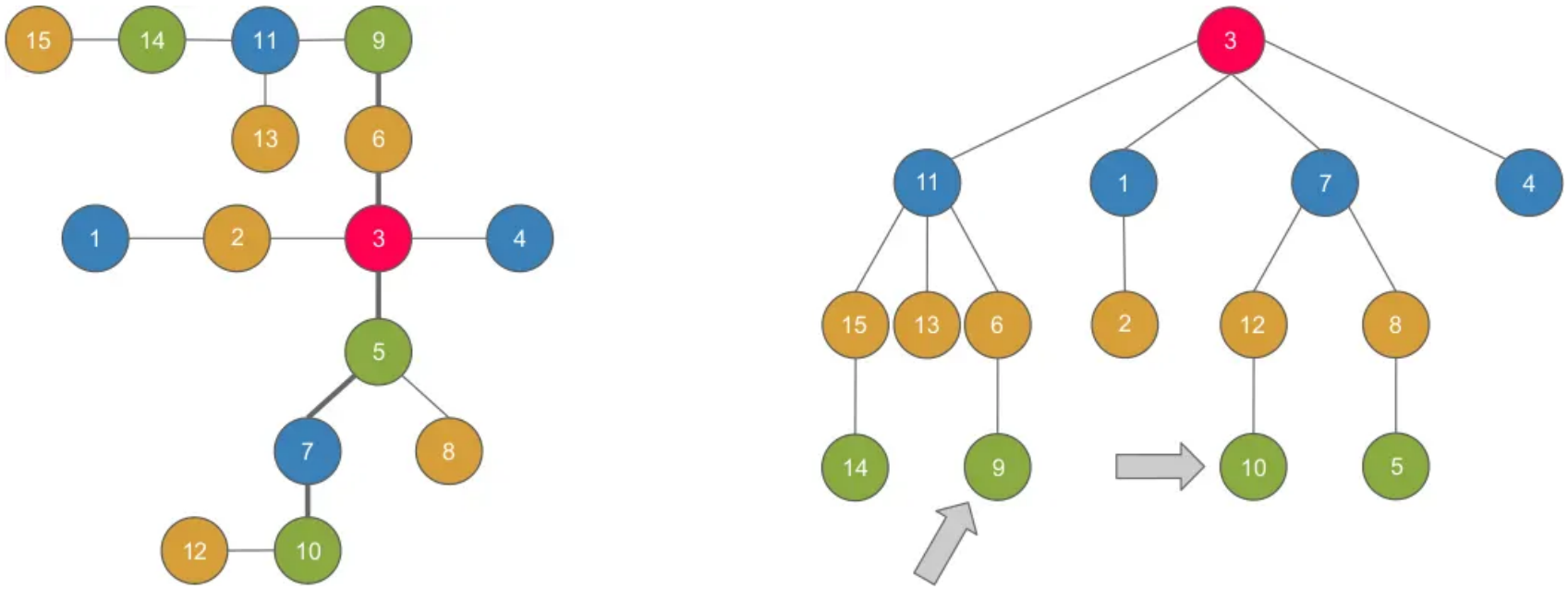


Példa: A 14-es csúcs a 14, 15, 11 és 3 csúcsok komponenseihez tartozik.

Tulajdonság 2: Utak felbontása

Állítás: Az eredeti fa bármely `a` és `b` csúcsa közötti út egyértelműen felbontható két részre a centroid fabeli legkisebb közös ősükön ($\mathtt{LCA}_{CD}$ `(a, b)`) keresztül.

- Az `a` $\rightarrow$ `b` út az eredeti fában áthalad az $\mathtt{LCA}_{CD}$ `(a, b)` csúcson.
- **Bizonyítás (indirekt):** Ha nem haladna át rajta, akkor `a` és `b` ugyanabban a komponensben maradna az $\mathtt{LCA}_{CD}$ `(a, b)` eltávolítása után, tehát lenne náluk mélyebb közös ősük a centroid fában.



Példa: A 9 és 10 közötti út az eredeti fában felbontható egy 9 $\rightarrow$ 3 és egy 3 $\rightarrow$ 10 útra (ahol 3 a közös centroid ős).

Tulajdonság 3: Az utak hatékony reprezentációja

Ez a legfontosabb tulajdonság a feladatmegoldáshoz:

Állítás: Az eredeti fa mind az N^2 darab útja előállítható összesen $O(N \log N)$ darab, "csúcstól az őséig" tartó út kombinációjaként.

Miért fontos ez?

- Bármely $a \to b$ út vizsgálata helyett elég minden csúcsra csak a centroid-fabeli **őseihez vezető utakat** tárolni.
- Mivel a fa magassága $O(\log N)$, minden csúcsnak csak ennyi őse van.
- N csúcs $\times$ $\log N$ ős = $O(N \log N)$ összes eltárolandó információ.

Általános megoldási séma: Minden centroidnál csak azokat az utakat vizsgáljuk, amelyek **áthaladnak** az adott centroidon. Mivel minden út pontosan egy centroidhoz fog tartozni (mint legmagasabb szintű centroid az úton), minden utat pontosan egyszer számolunk.

Heavy-Light Dekompozíció

Heavy-Light Dekompozíció (HLD)

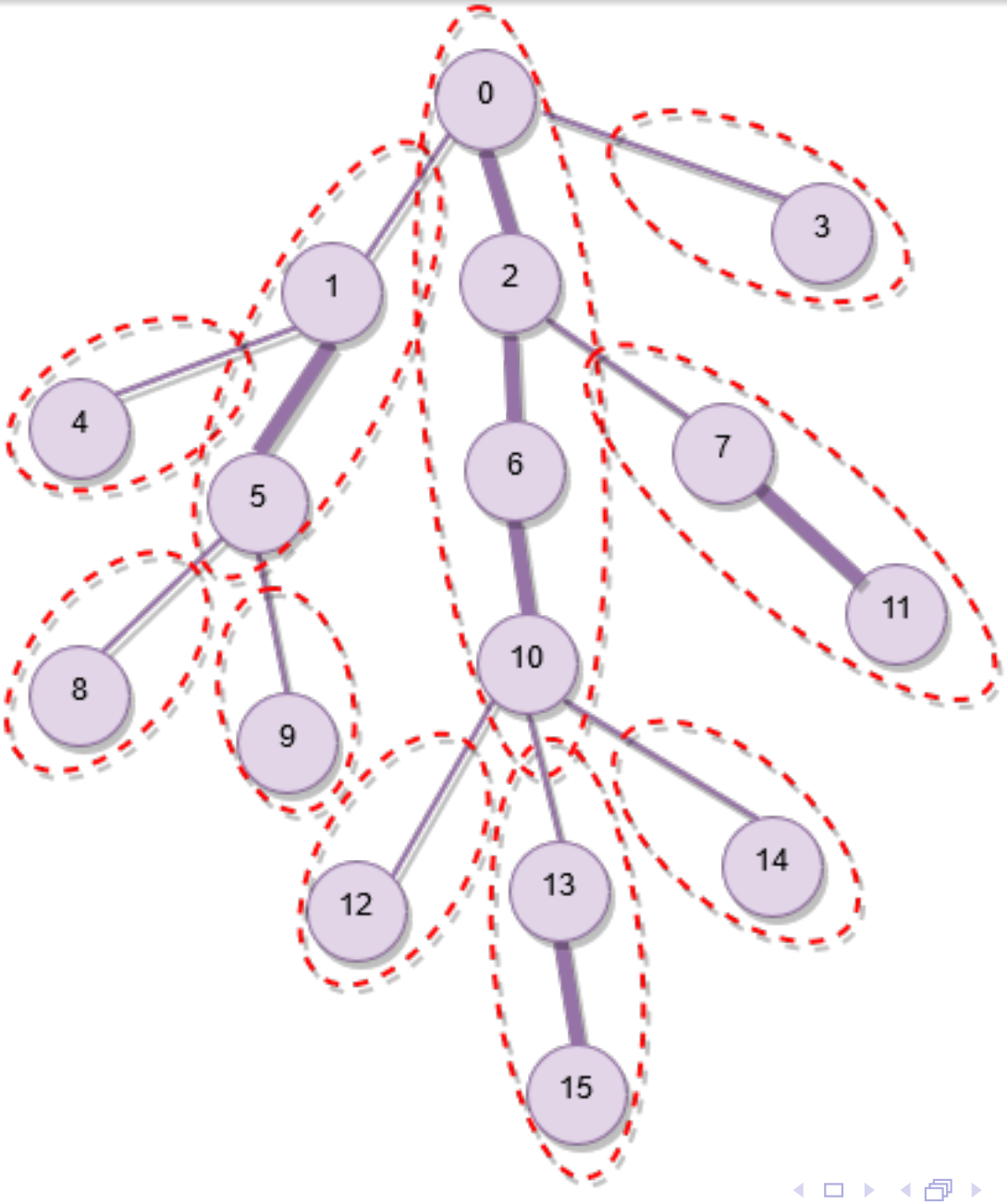
A **Heavy-Light Dekompozíció** egy olyan technika, amely a fát diszjunkt utakra bontja fel. Ezzel a fán végzett lekerdezéseket lineáris adatszerkezetek (pl. szegmensfa) lekérdezéseire vezethetjük vissza.

Az alapötlet:

- A fát felosztjuk több útra.
- Bármely `v` csúcsból a gyökérig tartó úton legfeljebb `O(log n)` utat kell kereszteznünk.
- Az utakat úgy választjuk meg, hogy ne metsszék egymást.

Haszna: Egy "számoljunk valamit `a` és `b` közötti úton" típusú kérdést lefordíthatunk `O(log n)` darab "számoljunk valamit egy szegmensfán `[l, r]` között" típusú kérdésre.

HLD példa



Konstrukció: Nehéz és könnyű élek

Jelölje $s(v)$ a v alatti részfa méretét. Tekintsük a v-ből lefelé induló éleket:

Nehéz él (Heavy edge): Olyan (v, c) él, ahol c a v gyermeke és:

$$s(c) \geq \frac{s(v)}{2}$$

Egy csúcsból lefelé **legfeljebb egy** nehéz él indulhat (különben $s(v) \geq 1 + 2 \cdot \frac{s(v)}{2} > s(v)$ lenne).

Könnyű él (Light edge): Minden más él, amely nem nehéz.

Nehéz út (Heavy path): Nehéz élekből álló maximális, diszjunkt utak. Minden csúcs pontosan egy nehéz úthoz tartozik (amely állhat egyetlen csúcsból is).

Helyesség és komplexitás

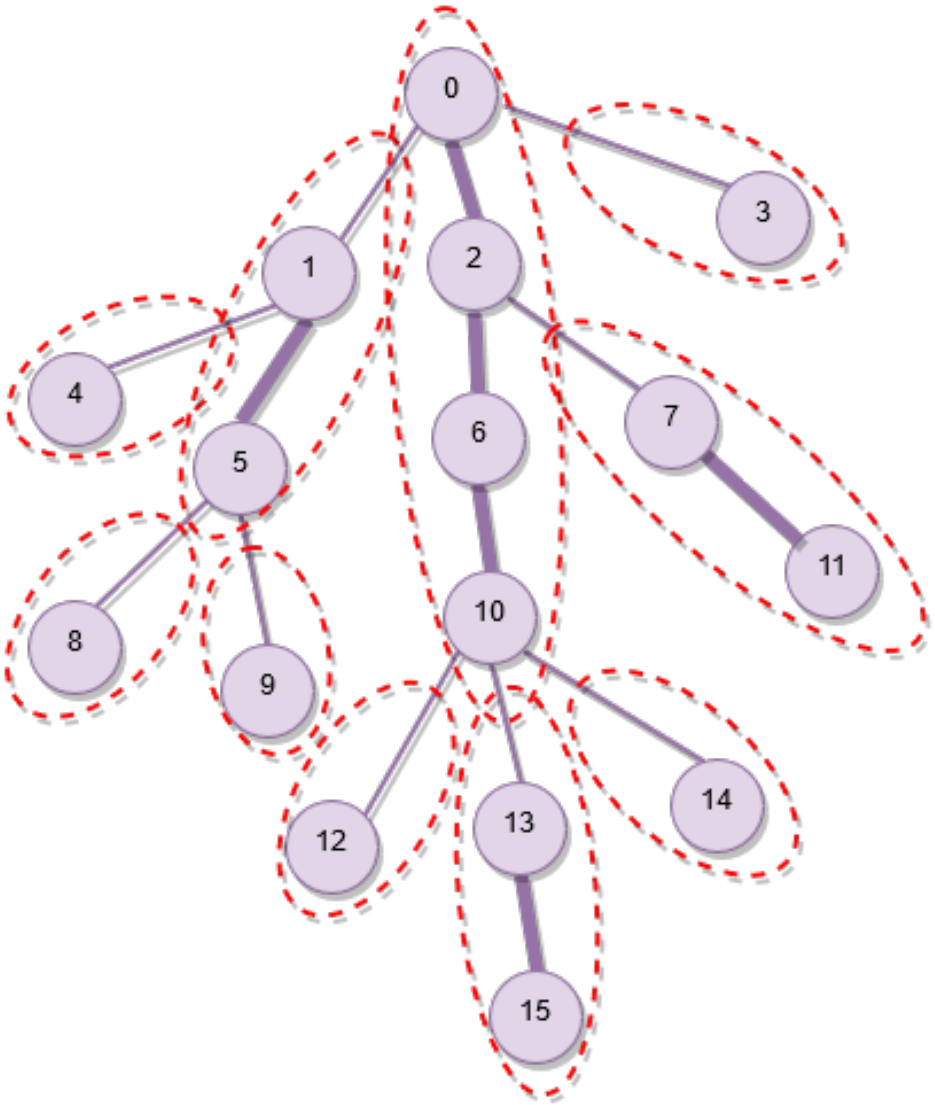
Tétel: A gyökértől bármely v csúcsig tartó úton legfeljebb $\log n$ darab könnyű élen haladunk át.

Bizonyítás:

- Ha egy könnyű élen (v, c) megyünk lefelé, akkor tudjuk, hogy $s(c) < s(v)/2$.
- Ez azt jelenti, hogy minden könnyű él legalább **felére csökkenti** a részfa méretét.
- n méretű fában legfeljebb $\log_2 n$ alkalommal tudjuk felezni a méretet, mielőtt 1-et érnénk el.

Következmény: Mivel két nehéz út között csak könnyű élen válthatunk, egy $a \to b$ úton legfeljebb $O(\log n)$ nehéz utat érintünk.

Szemléltetés



Jellemzők az ábrán:

A vastag élek a nehéz élek (heavy edges).

A szaggatott keretek jelölik a nehéz utakat (heavy paths).

Figyeljük meg, hogy a gyökértől bármely levélig tartó úton csak kevés "ugrás" (könnyű él) van az utak között.

Útlekerdezés példa

Tegyük fel, hogy minden csúcsnak van egy értéke, és a maximumot keressük az `a` és `b` közötti úton:

1. Keressük meg `a` és `b` legkisebb közös ősét (`l = LCA(a, b)`).
2. A feladat két útra bomlik: `(a, l)` és `(b, l)`.
3. Egy `(u, l)` úton haladva:
   - Megnézzük, melyik nehéz úton van `u`.
   - Lekérdezzük az szegmensfát `u` és az aktuális nehéz út teteje (vagy `l`) között.
   - Átnyergelünk a nehéz út feletti szülőre (könnyű él) és folytatjuk, amíg el nem érjük `l`-t.

Komplexitás:

- $O(\log n)$ útugrás $\times$ $O(\log n)$ szegmensfa művelet = $O(\log^2 n)$.
- (Ha az LCA keresést is az ugrásokba integráljuk, a logika ugyanez).

HLD Implementáció — Felépítés

```
1  vector<int> parent, depth, heavy, head, pos;
2  int cur_pos;
3
4  int dfs_sz(int v, vector<vector<int>> const& adj) {
5      int size = 1, max_c_size = 0;
6      for (int c : adj[v])
7          if (c != parent[v]) {
8              parent[c] = v, depth[c] = depth[v] + 1;
9              int c_size = dfs_sz(c, adj);
10             size += c_size;
11             if (c_size > max_c_size)
12                 max_c_size = c_size, heavy[v] = c;
13         }
14     return size;
15 }
16
17 void decompose(int v, int h, vector<vector<int>> const& adj) {
18     head[v] = h, pos[v] = cur_pos++;
19     if (heavy[v] != -1)
20         decompose(heavy[v], h, adj);
21     for (int c : adj[v])
22         if (c != parent[v] && c != heavy[v])
23             decompose(c, c, adj);
24 }
```

HLD Implementáció — Lekérdezés

```
1  int query(int a, int b) {
2      int res = 0;
3      for (; head[a] != head[b]; b = parent[head[b]]) {
4          if (depth[head[a]] > depth[head[b]]) swap(a, b);
5          // Lekerdezes az szegmensfan az aktualis uton
6          int cur_max = segment_tree_query(pos[head[b]], pos[b]);
7          res = max(res, cur_max);
8      }
9      // Az utolso szakasz, amikor mar egy uton vannak
10     if (depth[a] > depth[b]) swap(a, b);
11     res = max(res, segment_tree_query(pos[a], pos[b]));
12     return res;
13 }
```

Magyarázat:

- A `pos[v]` értékek úgy vannak kiosztva, hogy egy nehéz úton lévő csúcsok folytonos tartományt alkossanak az szegmensfában.
- Mindig a mélyebben lévő út fejét (`head`) ugratjuk feljebb.

Alkalmazások és variációk

Mivel minden csúcsot leképeztünk egy lineáris tömbre (`pos[v]`), bármilyen standard szegmensfa-feladat (akár lazy propagation-nel) megoldható fán is.

Példa:

- Egy fa minden éle kezdetben fehér. `(a, b, c)` formában érkeznek frissítések: az `a` – `b` út minden élét színezzük át `c`-re. Hány él van az egyes színekből a végén?

Élek kezelése: Ha a feladat az éleken tárol értékeket (nem a csúcsokon):

- Rendeljük minden él értékét a mélyebben fekvő végpontjához.
- Lekérdezésnél az LCA csúcsát ki kell hagyni (mivel az az LCA feletti élet reprezentálná).

HLD Alternatívák és rokon technikák

A fa-algoritmusok eszköztára széles; a HLD bár rendkívül rugalmas, nem mindig a leghatékonyabb vagy legegyszerűbb választás.

Binary Lifting:

- **Alkalmazás:** Statikus fák esetén (nincs érték-frissítés).
- **Előny:** Nagyon gyors, $O(\log n)$ lekérdezés és egyszerűbb kód. Ideális LCA keresésre vagy úton lévő maximum/összeg meghatározására, ha az adatok nem változnak.

Euler Tour Technique (ETT / DFS sorrend):

- **Alkalmazás:** Elsősorban **részfákat** érintő műveletekhez.
- **Logika:** A fát a DFS bejárás sorrendjében egy tömbre képezzük le. Egy részfa ebben a tömbben egyetlen folytonos intervallum $[start_v, end_v]$.
- **Összevetés:** Részfa-frissítésre és lekérdezésre gyorsabb ($O(\log n)$), mint a HLD, de az utak kezelésére (path queries) önmagában nem alkalmas.

Centroid Dekompozíció:

- **Alkalmazás:** Olyan globális statisztikákhoz, ahol az utak hosszát vagy egyéb tulajdonságait kell vizsgálni.
- **Különbség:** Nem szegmensfára épít, hanem a fa hierarchikus "szétvágására".

Feladatok

Fixed-Length Paths I – Feladat

`https://cses.fi/problemset/task/2080`

Adott egy fa n csúccsal és egy k egész szám.

Feladat: Számoljuk meg, hány olyan egyszerű út van a fában, amely pontosan k élet tartalmaz!

Korlátok:

- $1 \leq n, k \leq 2 \cdot 10^5$

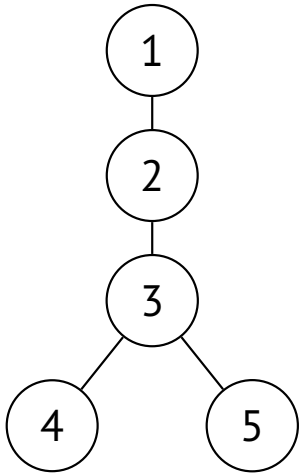
Bemenet: Az első sor tartalmazza n és k értékét. A következő $n - 1$ sor mindegyike egy-egy élet ír le (u, v).

Példa bemenet:

```
5 2
1 2
2 3
3 4
3 5
```

Kimenet:

4



Utak ($k = 2$):
1-2-3
2-3-4
2-3-5
4-3-5

Path Queries II – Feladat

https://cses.fi/problemset/task/2134

Adott egy fa n csúccsal, minden csúcshoz tartozik egy egész érték. q műveletet kell elvégeznünk. A Lekérdezések típusai:

- **1** s x**:** Az s csúcs értékét változtasd meg x-re.
- **2** a b**:** Add meg a maximális csúcsértéket az a és b közötti úton.

Korlátok:

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq v_i, x \leq 10^9$

Bemenet: Első sor: n, q. Második sor: n darab kezdőérték. Következő $n-1$ sor: élek. Végül q sorban a lekérdezések.

Példa bemenet:

```
5 3
2 4 1 3 3
1 2
1 3
2 4
2 5
2 3 5
1 2 4
2 3 5
```

Kimenet:

```
4 3
```

Minmaxtree – Feladat

https://oj.uz/problem/view/BOI18_minmaxtree

Adott egy N csúcsú fa, amelynek élsúlyai elvesztek. Ismerjük K különböző lánc (útvonal) mentén mért minimum vagy maximum értékeket. Minden megadott szélsőérték egyedi.

Feladat: Rendeljünk minden élhez egy olyan súlyt, amely konzisztens az összes megadott minimum és maximum feltétellel.

Korlátok:

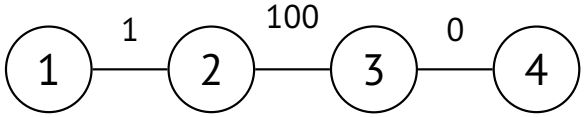
- $1 \leq N \leq 70\,000$
- $1 \leq K \leq 70\,000$
- Az élsúlyok (v) 32 bites előjeles egészek legyenek.

Bemenet: Az első sor N értékét tartalmazza[cite: 11]. A következő $N - 1$ sor a fa éleit (x, y) adja meg. Ezután következik K, majd a láncok: M x y z (maximum az $x - y$ úton z) vagy m x y z (minimum az $x - y$ úton z).

Kimenet: $N - 1$ sor, mindegyik három egész számot (x, y, v) tartalmaz, amely egy élet és a hozzárendelt súlyt jelöli.

Minmaxtree – Példa

https://oj.uz/problem/view/BOI18_minmaxtree



Példa bemenet:

```
4
1 2
2 3
3 4
3
M 1 2 1
m 1 4 0
M 2 3 100
```

Példa kimenet:

```
3 2 100
1 2 1
4 3 0
```

Konzisztencia ellenőrzés:
Max(1,2) = 1 ✓
Min(1,4) = 0 ✓
Max(2,3) = 100 ✓

Factories – Feladat

`https://oj.uz/problem/view/JOI14_factories`

Adott egy $\mathtt{N}$ városból álló ország, ahol a városokat $\mathtt{N} - 1$ út köti össze (fa szerkezet). Minden út hossza ismert. Különböző cégeknek több városban is lehetnek gyáraik.

Feladat: Dolgozzunk fel $\mathtt{Q}$ lekérdezést! Minden lekérdezésben két cég ($\mathtt{U}$ és $\mathtt{V}$) gyárainak listáját kapjuk meg. Határozzuk meg a két cég gyárai közötti minimális távolságot!

Cél: Keressük meg a legkisebb $\mathtt{dist(u, v)}$ értéket, ahol $\mathtt{u} \in \{\mathtt{U}$ cég gyárai$\}$ és $\mathtt{v} \in \{\mathtt{V}$ cég gyárai$\}$.

Korlátok:

- $2 \leq \mathtt{N} \leq 500\,000$
- $1 \leq \mathtt{Q} \leq 100\,000$
- Az utak hossza $\mathtt{D_i} \leq 10^8$
- A gyárak számának összege az összes lekérdezésre $\leq 1\,000\,000$.

Factories – Példa

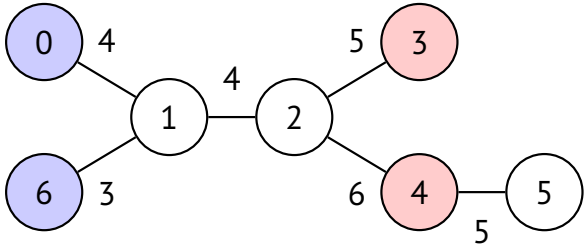
https://oj.uz/problem/view/JOI14_factories

Példa bemenet:

```
7 3
0 1 4
1 2 4
2 3 5
2 4 6
4 5 5
1 6 3
2 2
0 6
3 4
3 2
0 1 3
4 6
1 1
2
5
```

Kimenet:

```
12
3
11
```



Első lekérdezés (fenti ábrán):

U = {0, 6}
V = {3, 4}
dist(6, 3) = 3 + 4 + 5 = **12**

További feladatok

IOI 2011 Race
https://oj.uz/problem/view/IOI11_race

JOI 2013 Synchronization
https://oj.uz/problem/view/JOI13_synchronization

Kódkupa 2022 Három különböző
https://njudge.hu/problemset/main/KK22_kulonbozo3/

JOI 2018 Cats or Dogs
https://oj.uz/problem/view/JOI18_catdog